

TITANIC SURVIVAL PREDICTION

Machine Learning Pipeline — Architettura Avanzata con Libreria Generalizzata

5 Algoritmi | Docker | REST API | Feature Engineering | Libreria Riutilizzabile

5

Modelli

891

Train rows

418

Test rows

84.4%

Best CV acc.

Indice

Struttura della presentazione

01

Il Problema

Obiettivo Kaggle e analisi del dataset

02

Architettura

Docker, REST API, config.yaml

03

Libreria Generalizzata

Abstract classes, modularità, riuso

04

Data Pipeline

Sequenza completa dei transformer

05

Feature Engineering

7 nuove variabili derivate

06

Imputazione Age

Strategia per titolo del nome

07

Modelli & GridSearchCV

5 algoritmi con ottimizzazione

08

Tecniche Avanzate

PCA, Multicollinearity, Threshold Tuning

09

Parallelismo & Docker

Fix deadlock, n_jobs

10

Valutazione Locale

StratifiedKFold vs holdout

11

Risultati

Performance CV per modello

12

Prossimi Passi

Ensemble, CV-Bagging, roadmap

Il Problema — Titanic Kaggle

Binary classification: Survived = 0 / 1

Obiettivo

Predire la sopravvivenza di 418 passeggeri del test set in base a caratteristiche demografiche e sociali. Kaggle valuta con Accuracy (% predizioni corrette).

Dataset Kaggle

891 passeggeri nel train set (labels noti)

418 passeggeri nel test set (labels segreti)

Metrica Kaggle: Accuracy

Baseline naive (tutti morti): 61.6%

Obiettivo realistico: >80%

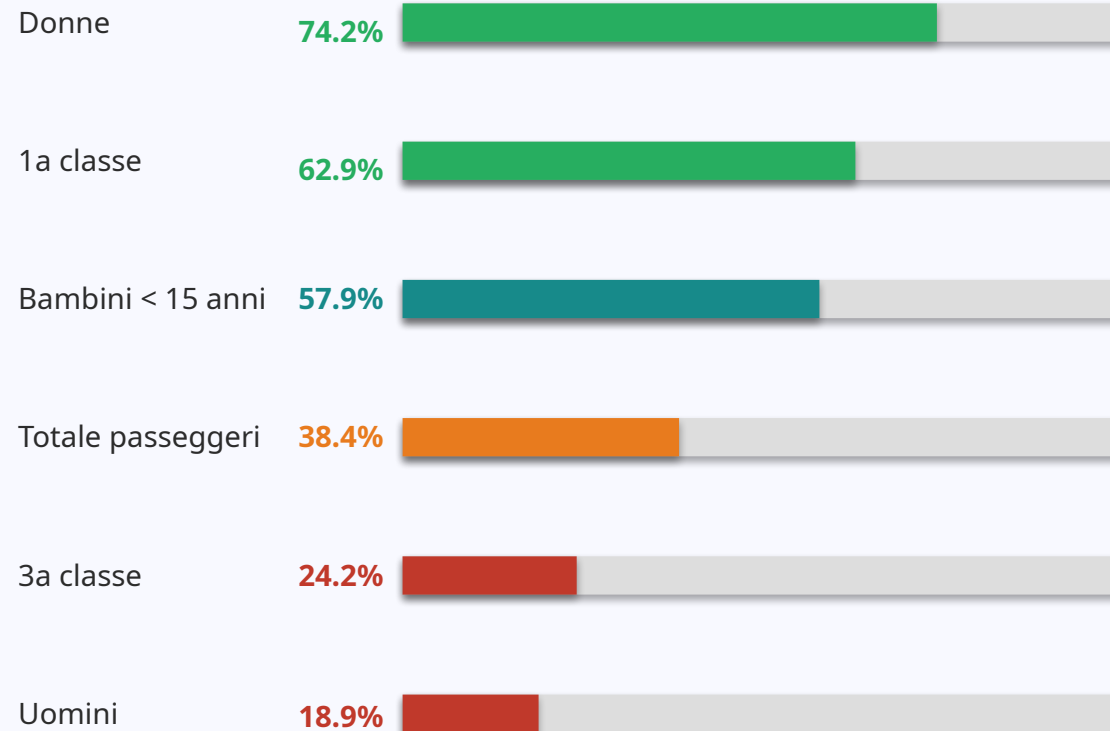
Feature originali (11 colonne)

PassengerId, Pclass (1/2/3), Name

Sex, Age (177 mancanti), SibSp, Parch

Ticket, Fare, Cabin (687 mancanti), Embarked

Tassi di sopravvivenza (train set)



Architettura del Progetto

Docker | REST API | config.yaml | Libreria embedded

titanic/

```
| - lib/
|   | - models/
|   |   | - base.py
|   |   | - logreg.py
|   |   | - random_forest.py
|   |   | - xgboost.py
|   |   | - svm.py
|   | +- decision_tree.py
|   | - api/models.py
|   | - feature_engineering.py
|   | - cleaner.py
|   | - encoder.py
|   | - outliers.py
|   | - multicollinearity.py
| +- pca_reducer.py
|- main.py
|- config.yaml
|- Dockerfile
|- evaluate_local.py
+- tests/
```

Docker Container

```
FROM python:3.11-slim

ENV PYTHONPATH=/app:/app/lib

ENV PYTHONUNBUFFERED=1

ENV LOKY_MAX_CPU_COUNT=8

RUN apt install gcc (per XGBoost)
```

REST API -- Flask

```
GET /      -> lista modelli attivi

POST /train -> training + best_score

POST /evaluate -> accuracy, f1, report

POST /predict -> genera submission CSV

Risposta JSON strutturata

Avvio: bash start.sh nel container
```

config.yaml -- punto di controllo

loader:

```
train_path, target_col
drop_cols: [Ticket]
```

cleaner:

```
num_strategy: knn
cat_strategy: knn-----
```

outliers:

```
detection: [iqr, zscore]
apply_to: [logreg]
```

multicollinearity:

```
threshold: 0.85
apply_to: [logreg]
```

pca:

```
n_components: 0.95
apply_to: [logreg]
```

models:

```
active: [logreg, xgboost,
rf, dt, svm]
```

model_cache:

```
enabled: false
```

Libreria Generalizzata -- lib/

Componenti riutilizzabili su qualsiasi dataset tabellare, non solo Titanic

La cartella lib/ non e' specifica per il Titanic -- e' una libreria ML generica. Ogni componente prende un DataFrame in input, lo trasforma, e restituisce un DataFrame indipendentemente dal dominio. Per usarla su un nuovo dataset basta cambiare config.yaml.

models/base.py -- AbstractModel

```
class AbstractModel(ABC):
```

```
    def __init__(self):
```

```
        self.model = None
```

```
        self.threshold = 0.5
```

```
        self.best_score = None
```

```
        self.best_params = None
```

```
    @abstractmethod
```

```
    def train(self, X, y, cv, scoring):
```

```
        ...
```

```
    def predict(self, X):
```

```
        if self.threshold != 0.5:
```

```
            p = model.predict_proba(X)[:,1]
```

```
            return (p >= self.threshold)
```

```
        return self.model.predict(X)
```

GenericCleaner

Cosa fa: Riempie i valori mancanti (NaN) nel dataset. Impara la strategia di imputazione sul train, e poi applica la stessa logica al test set. Supporta anche KNN:

GenericOutlierHandler

Identifica e corregge i valori anomali (outlier) nelle colonne numeriche. Sul training set calcola i limiti oltre i quali un valore è considerato anomalo (usando IQR, Z-score, o entrambi).

MulticollinearityHandler

Individua coppie di feature numeriche che portano la stessa informazione (correlazione di Pearson > soglia, es. 0.85). Per ogni coppia ridondante, rimuove automaticamente quella con la correlazione media più alta con tutto il resto

GenericEncoder

Converte il DataFrame in un formato puramente numerico che i modelli sklearn possono leggere. Per le colonne numeriche applica StandardScaler (media 0, deviazione 1). Per le colonne categoriche applica One-Hot Encoding

PCAReducer

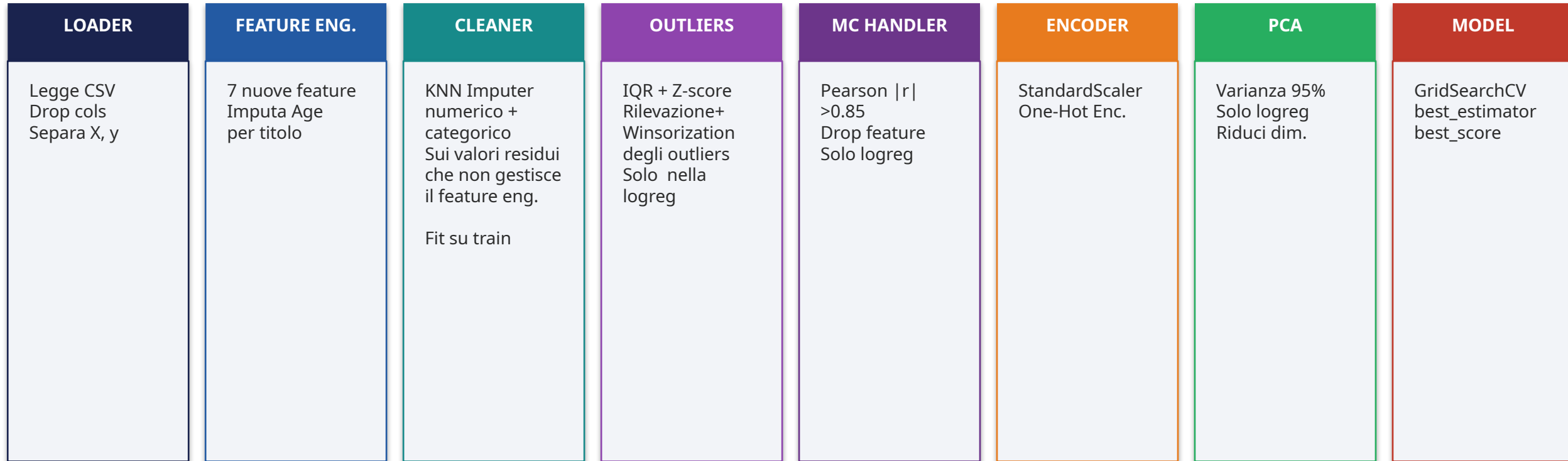
Riduce il numero di colonne combinando quelle esistenti in "componenti principali" che catturano la massima varianza con il minor numero di dimensioni. Si configura con una soglia di varianza (es. 0.95 = mantieni il 95% dell'informazione); il numero di componenti risultante è automatico.

FeatureEngineer

Crea nuove colonne derivate dalla conoscenza del dominio Titanic. Sul training set "impara" (es. la mediana dell'età per ogni gruppo sociale) e poi applica la stessa trasformazione al test set senza guardarlo.

Data Pipeline

Dal CSV grezzo ai vettori numerici pronti per i modelli



Regola anti-leakage: ogni transformer viene fittato ESCLUSIVAMENTE sui dati di train, poi applicato al test con transform().

L'encoder viene fittato su un numero fisso di colonne: se il MulticollinearityHandler ne rimuove una, serve un'istanza separata di GenericEncoder (e PCAReducer) già fittata sul dataset ridotto, altrimenti sklearn darebbe errore.

Feature Engineering -- Le Variabili Derivate

7 nuove feature costruite sulla conoscenza del dominio Titanic

FarePerPerson

Biglietti condivisi tra famiglie hanno tariffa unica. La tariffa pro-capite riflette la classe economica individuale, non il totale del gruppo.

Imputazione Age per titolo

i 177 valori Age mancanti vengono stimati con la mediana del gruppo (Master $\approx$ 5 anni, Mr $\approx$ 30, Mrs $\approx$ 36) invece della mediana globale ($\sim$ 28)

AgeGroup

Fascia d'età (Child, Teen, Adult, Middle, Senior)

FamilySize IsAlone

Dimensione del nucleo familiare a bordo. Famiglie medie (2-4) avevano maggiore probabilita' di sopravvivenza rispetto a solitari o famiglie numerose. I solitari avevano dinamiche di sopravvivenza diverse.

CabinKnown/ CabinDeck

Avere una cabina registrata era correlato con la classe sociale. Solo i passeggeri di 1a classe avevano generalmente la cabina assegnata.

SocialClass

Combina classe e genere: '1st_female', '3rd_male' ecc. Le donne di 1a classe sopravvivevano al 97%, gli uomini di 3a al 15%.

TitleGroup

Il titolo (Mr, Mrs, Miss, Master, Rare) porta informazioni su sesso, età e status sociale. Master = bambino maschio, alta sopravvivenza.

Usata anche per imputazione Age

Feature Engineering -- Imputazione Age per Titolo

177 valori Age mancanti (19.9%) imputati con mediana per gruppo sociale

Il problema: 177 Age mancanti su 891 (19.9%)

Imputare con la mediana globale (~28 anni) sbaglia sistematicamente: assegna 28 anni a 'Master Smith' (bambino di 3 anni) e a 'Mrs. Brown' (anziana).

La soluzione: mediana per TitleGroup (fit su train)

Master	~5 anni	Bambini maschi (figlio, nipote...)
Miss	~22 anni	Donne non sposate, spesso giovani
Mrs	~36 anni	Donne sposate / madri
Mr	~30 anni	Uomini adulti (la maggioranza)
Others	mediana gruppo	Rev, Dr, Col, Major, Lady...

feature_engineering.py

```
_TITLE_MAP = {
    'Master': 'Master',
    'Miss': 'Miss', 'Ms': 'Miss',
    'Mrs': 'Mrs', 'Mme': 'Mrs',
    'Mr': 'Mr', 'Sir': 'Mr', ...
} # tutti gli altri -> 'Rare'

def fit(self, df):
    titles = df['Name'].str.extract(
        r'([^\.\+]\.?)?[0]
    grp = titles.map(_TITLE_MAP)
        .fillna('Rare')
    tmp = df.assign(TitleGroup=grp)
    self._age_medians = (
        tmp.groupby('TitleGroup')
            ['Age'].median()
        .to_dict()
    )
    self._fallback = df['Age'].median()
```

Fallback: se un gruppo non compare nel train (es. Rare vuoto), si usa la mediana globale del train -- nessun NaN residuo dopo la fase di fit.

Modelli & GridSearchCV

5 algoritmi con iperparametri ottimizzati tramite cross-validation interna

Logistic Regression

Calcola una combinazione lineare delle feature e la converte in probabilità. Semplice e interpretabile, ma funziona bene solo se le relazioni con il target sono lineari.

Decision Tree

Divide i dati in rami con domande sulle feature ("età < 18?") fino ad arrivare a foglie pure. Molto interpretabile ma tende all'overfitting senza limitare la profondità.

Random Forest

Costruisce centinaia di alberi su sottoinsiemi casuali di dati e feature, poi fa il voto di maggioranza. La casualità riduce l'overfitting: modello più robusto del singolo albero.

XGBoost

Costruisce alberi in sequenza: ogni albero corregge gli errori del precedente. Più preciso ma più sensibile agli iperparametri rispetto al Random Forest.

SVM

Cerca il confine che massimizza il margine tra le due classi. Efficace su dataset piccoli con feature ben scalate.

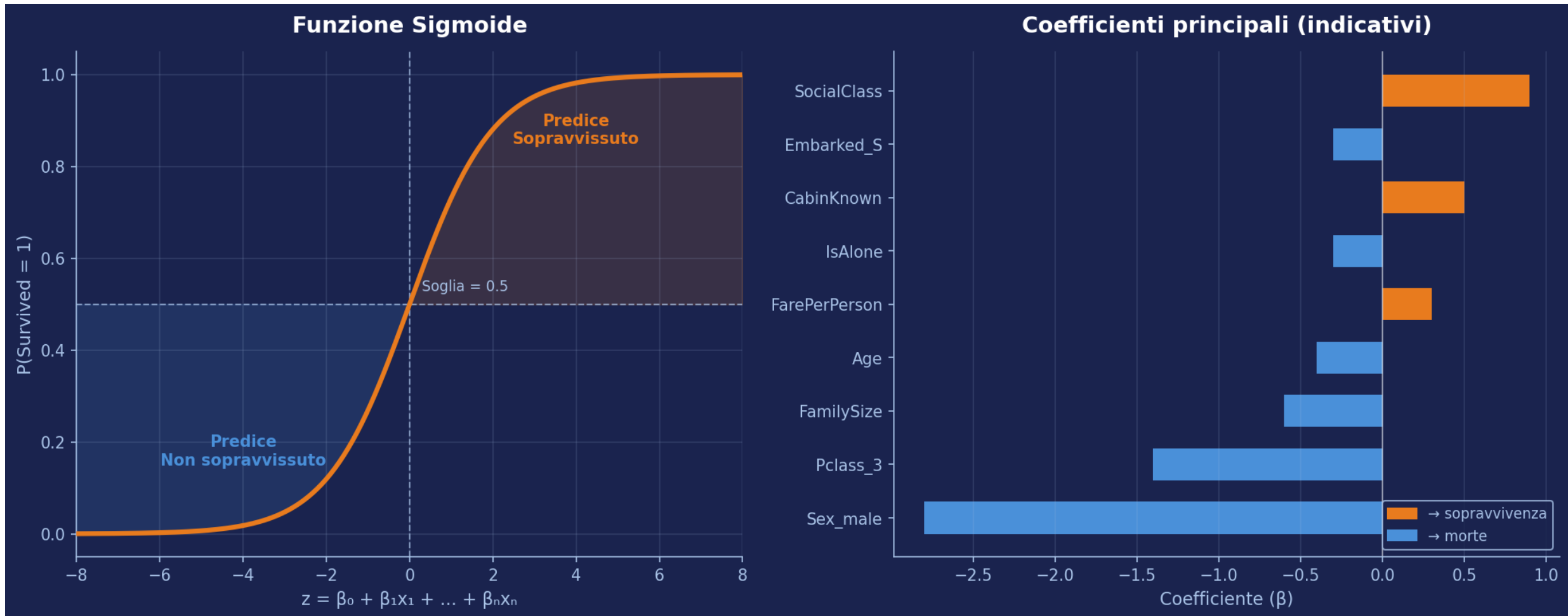
Ensemble di Modelli (Soft Voting)

Invece di scegliere un solo modello, combina le probabilità di più modelli e fa la media pesata. Ogni modello "vota" quanto è probabile che un passeggero sia sopravvissuto, i voti vengono sommati con pesi diversi (xgboost, logreg, svm) e la predizione finale è quella con la probabilità media più alta. Riduce gli errori individuali dei singoli modelli: dove uno sbaglia, un altro può compensare.

GridSearchCV: cv=5, scoring='f1_weighted', n_jobs=-1 (parallelo a livello di fold).
best_score_ e' la stima onesta della cross-validation interna, non calcolata sul train.
best_estimator_ viene poi ri-valutato con cross_val_score(scoring='accuracy') per ottenere la stima Kaggle.

Logistic Regression — Curva Sigmoidale e Coefficienti

Come il modello trasforma una combinazione lineare di feature in una probabilità



Risultati — Performance CV Locale (Nuovo Seed)

StratifiedKFold(5), scoring='accuracy' su train.csv (891 campioni)

Modello	Dataset	Accurac y	+/- Std	Bar
Ensemble	xgboost + logreg + svm	0.8440	±0.0130	
XGBoost	orig	0.8417	±0.0181	
Random Forest	orig	0.8350	±0.0077	
SVM	orig (no Fare)	0.8339	±0.0132	
Logistic Reg.	win+no-mc (no Fare)	0.8294	±0.0136	
LogReg + PCA	win+no-mc (no Fare)+PCA	0.8193	±0.0096	
Decision Tree	orig	0.8148	±0.0248	

Miglior modello: Ensemble → Accuracy=0.8440 (stima Kaggle: ~84.4%) | Ensemble funziona! | RF più stabile: std 0.0077

Analisi dei risultati

Ensemble migliore 84.4%

Con il nuovo seed il voting funziona. XGBoost+LogReg+SVM si complementano e superano ogni singolo modello.

XGBoost secondo 84.2%

Solo 0.2 pp sotto l'ensemble. Con std 0.0181 è leggermente meno stabile di RF.

RF più stabile: std 0.0077

La deviazione standard più bassa di tutti. Modello robusto e consistente tra i fold.

Decision Tree più instabile

Std 0.0248 la più alta del gruppo — overfitting evidente senza bagging o ensemble.

LogReg + PCA peggior

PCA perde informazione su questo dataset piccolo (891 righe). Confermato anche con seed diverso.

Prossimi Passi -- Ensemble & Roadmap

Come superare l'84% e raggiungere le performance dei notebook di riferimento

Soft Voting Ensemble

VotingClassifier(voting='soft') su logreg + xgboost + random_forest.
Media delle probabilita' dei 3 modelli migliori.
Il notebook di riferimento raggiunge ~84% con questo approccio.
Implementazione: 5 righe con sklearn VotingClassifier.

Guadagno atteso: +1-2 pp su 84.4%

CV-Bagging su Random Forest

Approccio dell'articolo leaderboard (~87%).
Training del modello su ogni fold di StratifiedKFold(5).
Predizioni test = media delle probabilita' dei 5 modelli.
RF 1750 alberi, max_depth=7 -> piu' espressivo del nostro.

Guadagno atteso: +2-3 pp, submission piu' stabile

Stacking con Meta-learner

Modelli base: RF, XGBoost, SVM -> predizioni OOF come feature.
Meta-learner: LogisticRegression sulle predizioni base.
Attenzione al leakage: base e meta su split separati.
Implementazione con sklearn StackingClassifier.

Guadagno atteso: +1-2 pp, complessita' maggiore

Feature Interaction

Sex x Pclass: feature di interazione esplicita.
TicketFreq: quante persone condividono il ticket.
AgeGroup: fasce eta' (Child <12, Teen, Adult, Senior).
Fare binning: fasce tariffarie invece di valore continuo.

Guadagno atteso: +0.5-1 pp

Prossimi Passi — Roadmap

Ensemble implementato (84.4%) — come superare ulteriormente le performance

Stacking con Meta-learner

Modelli base: RF, XGBoost, SVM -> predizioni OOF come feature.
Meta-learner: LogisticRegression sulle predizioni base.
Attenzione al leakage: base e meta su split separati.
Implementazione con sklearn StackingClassifier.

Guadagno atteso: +1-2 pp, complessita' maggiore

Rete Neurale

Modello ispirato al cervello umano: neuroni collegati in strati che imparano pattern complessi. Cattura relazioni non lineari che gli alberi e la regressione non riescono a modellare. Richiede normalizzazione dei dati e tuning dei parametri (strati, neuroni, learning rate). Guadagno atteso: variabile, dataset piccolo penalizza

Guadagno atteso: +1-2 pp, complessita' maggiore

Grazie

Titanic Survival Prediction -- ML Pipeline Avanzata con Libreria Generalizzata

Docker

Containerizzazione

FlaskAPI

REST API

AbstractModel

Libreria generalizzata

GridSearchCV

Ottimizzazione HP

PCA + MC Handler

Riduzione dim.

StratifiedKFold

Valutazione onesta

Ensemble 84.4%

Best model

Feature Eng. x8

Nuove variabili